Scheme Saathi: Project Blueprint

A WhatsApp-native agent that finds the welfare scheme a person qualifies for, checks their paperwork before they apply, and decodes why a payment is stuck.

Version 1.0 · Hackathon track PS-01 · Prepared 19 October 2026

---

1. Executive summary

India has a very large number of central and state welfare schemes, and eligible people still miss out. The common explanation is "people don't know about schemes." The evidence says the problem is wider: people also fail on **paperwork consistency** and get **silently stuck after applying**.

Scheme Saathi is an agent that lives in WhatsApp, accepts Hindi or Marathi voice notes and text, and does three jobs:

1. **Eligibility and sequencing:** a rules engine finds the schemes a person qualifies for and recommends *one* to start with.
2. **Pre-flight check:** before the person applies, the agent compares names, dates of birth and linked records and flags mismatches that commonly cause rejection.
3. **Status decoder:** the person reports a portal status ("Payment Failed", "Rejected", "eKYC pending"), and the agent maps it to likely causes and the exact next action.

Nothing is submitted without an explicit, hash-verified human confirmation.

Core design principle: *the LLM talks, the rules engine decides.* The LLM handles conversation, extracts the profile from messy speech and explains results. Eligibility, mismatch detection and status mapping are deterministic code with sources.

---

2. The problem, in detail

2.1 Three failures, not one

Failure 1: Discovery. People don't know what they are entitled to. In a CEGA pilot in Delhi, migrant families were eligible for 20+ entitlements but were often unaware and had not applied. Portals such as myScheme are searchable but give little guidance on what to do next.

Scheme overload is a design constraint. The same pilot found that telling migrants about a *single* scheme (benefits, documents, next steps) led to roughly one extra application on average, while the multi-scheme arm showed signs of overload. A bot that dumps twelve matches may cause fewer applications than one that says "start with this one."

Failure 2: Readiness. Eligible people fail on documents and data consistency. An Indus Action study of Karnataka's guarantee schemes points to incomplete documentation and process issues on the citizen side, such as inactive bank accounts.

PM-KISAN shows the pattern clearly. Money reaches the farmer only when a chain of records agree: e-KYC done, land records verified, and the bank account mapped to Aadhaar in the NPCI mapper. Small mismatches (a spelling variant, a misplaced initial) can block payment indefinitely. NPCI seeding is separate from e-KYC: finishing e-KYC does not complete it.

Failure 3: Post-application silence. As of July 2025, more than 48 lakh of 10.64 crore registered PM-KISAN beneficiaries had not completed eKYC and had payments withheld, according to reporting by The Wire. The same report describes conflicting entries (one showing eKYC done, another showing not done) and statuses reverting after verification drives. A LibTech analysis in Andhra Pradesh found close to 30 exclusion reasons in the PM-KISAN MIS, with overlap between them. A farmer can have several blockers at once, and the portal rarely says which.

2.2 Reframed problem statement

> An eligible person doesn't need a list of schemes. They need the **one next action** that gets money into their account, and an explanation when it stalls.

2.3 Who has this problem (composite personas, illustrative)

Persona	Situation	What Saathi does
Small farmer	Payments stopped; Aadhaar says "Sunil", land record says "Suneel"	Flags the mismatch, says where to correct it, tracks follow-up
Widow	Likely eligible for a pension, unaware, inactive bank account	Finds the scheme, adds "reactivate account" as step 1
Migrant family	Eligible for 20+ entitlements, needs 2 or 3	Sequences by value and effort, one at a time

Persona	Situation	What Saathi does
CSC operator	Dozens of walk-ins a day, rules from memory	Operator dashboard, case tracking, summaries

---

3. Solution overview

3.1 The three layers

Layer 1: Eligibility and sequencing

- Asks the fewest questions needed (target: 5 to 7), chosen to eliminate the most uncertainty.
- Returns *eligible*, *near-miss* ("you miss X by ₹8,000 of income") and *ineligible* with the triggering rule.
- Recommends **one** scheme first, ranked by benefit value and application effort; queues the rest.
- Household mode: one conversation can cover several family members.

Layer 2: Pre-flight check (key differentiator)

- The user gives names and dates of birth as printed on each document (Aadhaar, bank passbook, land record), by text or by photo-assisted entry.
- Fuzzy matching with Hindi/Marathi transliteration normalisation flags probable mismatches.
- Checks the dependency chain for the scheme (e-KYC, NPCI seeding, active account, land seeding).
- Output: "Fix these 2 things first, and here is where to go."

Layer 3: Status decoder and unblocker

- User reports the portal status; the agent maps it to ranked likely causes, asks at most one diagnostic question, and gives a specific fix.
- Drafts a grievance note, schedules follow-up reminders, and can hand a case summary to a CSC operator.

3.2 Human confirmation (hard guarantee)

- `submit_application` is not callable by the LLM directly.

- The system shows a plain-language summary and asks for an explicit reply (for example, "1 to confirm, 2 to edit, 3 to cancel").
- The confirmation record stores a SHA-256 hash of the exact payload shown. If the payload changes, the hash fails and reconfirmation is required.
- Consent and confirmation are stored in the audit log.

3.3 Out of scope (be honest in the pitch)

- No integration with real government portals. "Submission" produces a pre-filled packet (PDF/JSON) and, optionally, a mock portal.
- No storage of full Aadhaar numbers.
- Causes shown are *likely* causes, not certainty; official grievance channels are always referenced.

---

4. Example conversation (Marathi/Hindi mixed, translated)

> **User (voice note):** "My PM-KISAN money stopped coming." **Saathi:** "Sorry to hear that. Let me check. What does the PM-KISAN portal show next to your name: Payment Failed, Rejected, or eKYC pending?" **User:** "eKYC pending." **Saathi:** "Got it. Two quick questions. Is the name on your Aadhaar and on your 7/12 land record spelled the same? Please type both." **User:** "Aadhaar: Sunil Patil. 7/12: Suneel Patil." **Saathi:** "These differ by one letter, and this is a common cause of blocked payments. Step 1: ask the talathi office to correct the 7/12 spelling, or update Aadhaar. Step 2: finish eKYC at a CSC. Step 3: ask your bank branch to confirm NPCI seeding. Shall I remind you in 5 days to check? Reply 1 for yes."

---

5. System architecture

```
User (WhatsApp text / voice note)
      |
   WAHA (WhatsApp HTTP API) ──webhook──▶ Backend API (FastAPI)
                                          ├─ Webhook handler (dedupe, ack
fast, enqueue)
                                          ├─ Speech layer: audio → STT
service → text
                                          ├─ Agent loop (LLM + tool calling)
                                          ├─ Tools: eligibility · preflight
· checklist ·
                                          |         prefill · followup ·
decode_status · submit(gated)
                                          ├─ Worker/scheduler (follow-ups,
retries)
                                          └─ Postgres · Redis · File volume
      ▲
   Replies via WAHA send API            Vercel website (Next.js) ──▶ Backend
API
```

5.1 Message lifecycle

1. User sends a message; WAHA emits a webhook to the backend over the private Docker network.
2. Backend verifies the request, deduplicates by message ID (Redis), returns 200 immediately, and enqueues the job.
3. Worker takes a per-user lock, loads state from Postgres, and, if the message is audio, downloads it, converts with ffmpeg and calls the STT service.
4. The agent loop runs: the LLM decides which tools to call; the tools execute deterministic logic; results go back to the LLM for a short, plain reply.
5. The reply (with a read-back of key facts) is sent through WAHA, with a small randomised delay.
6. Every tool call and its result is written to `audit_log`.

---

6. Technology stack

Layer	Choice	Why
WhatsApp gateway	**WAHA** (`devlikeapro/waha`)	Self-hosted REST API, quick to set up, no template approval
Backend	Python **FastAPI**	Fast to build, strong NLP/ASR ecosystem
Agent / LLM	Claude Sonnet 5.5 (primary), benchmark against a Gemini model	Reliable tool calling; test Hindi/Marathi quality
Rules engine	YAML scheme files + Python evaluator	Deterministic, testable, sourced
Name matching	`rapidfuzz` + custom transliteration normaliser	Handles spelling variants
STT	Self-hosted **AI4Bharat IndicConformer** (primary), **Sarvam** (fallback)	No quota dependence; audio stays on our server
TTS (optional)	Sarvam Bulbul v3	Hindi and Marathi voice replies
Database	**PostgreSQL** (JSONB for profiles)	Single source of truth
Cache/queue	**Redis** (+ RQ or Celery)	Dedupe, locks, jobs
Scheduler	APScheduler or RQ-scheduler	Follow-up reminders
PDF generation	WeasyPrint or ReportLab	Pre-filled form packets
Website	**Next.js** on **Vercel**	Landing page, operator dashboard, transparency page
Hosting	**GCP VM + EasyPanel**	Docker-based PaaS with domains and SSL
Tests	pytest, a persona-based eval harness	Accuracy numbers for judges
Observability	Structured JSON logs, audit table	Traceability

---

7. WAHA: decision record and risk controls

Decision: use WAHA for the WhatsApp channel, deployed in EasyPanel.

What WAHA is: an open-source Docker service that exposes WhatsApp as an HTTP API. It is **unofficial**: it automates a normal WhatsApp account rather than using Meta's Business Platform. Public WAHA material lists account ban risk for this approach, versus none for the official Cloud API.

Facts that shape the design (verify in WAHA docs before building):

- The free image supports a **single session**. Multiple sessions, the GOWS engine and media storage features are in WAHA Plus (paid key).
- Two engines are available in the free image: `WEBJS` (headless Chromium, heavier) and `NOWEB` (websocket-based, lighter). Start with NOWEB; fall back to WEBJS if a feature is missing.
- Login is by QR scan from a phone. **Persist the session folder on a volume** so a restart does not require a rescan.
- There are no WhatsApp templates or 24-hour windows here. That is convenient but removes Meta's guardrails.

Risk controls:

1. Use a **dedicated SIM/number**, never a personal one.
2. Only message people who messaged first. No cold outreach.
3. Keep proactive reminders few and low-volume; add random 1 to 3 second send delays and typing indicators.
4. Use **numbered text replies** ("Reply 1 to confirm") instead of interactive buttons, which are unreliable on unofficial gateways.
5. Protect WAHA's dashboard and API with an API key and basic auth; never expose it publicly without them. Confirm exact environment variable names in the WAHA docs.
6. Wrap WAHA behind a `WhatsAppGateway` interface (`send_text`, `send_document`, `download_media`) so it can be swapped for Meta's Cloud API without touching agent code.
7. Confirm how voice-note media is delivered in the free image (media URL or base64 in the webhook payload) in the first hour of the build.
8. Say in the deck that WAHA is the prototype channel and the production path is the official Cloud API with a verified business account.

---

8. Component specifications

8.1 Agent loop

- One system prompt: plain, short sentences; never decide eligibility; always call tools for rules; always read back numbers and names; reply in the user's language; never ask for Aadhaar numbers.
- Conversation state (profile, current scheme, step) is stored in Postgres, not just in the prompt.
- Keep tool schemas small and strict (JSON schema). Reject invalid tool arguments in code.
- Hard cap on tool iterations per message (for example, 6).

8.2 Tools

Tool	Purpose	Notes
`update_profile(member_id, fields)`	Save extracted facts	Validates types and ranges
`next_best_question(profile)`	Pick the question that prunes the most schemes	Scores by remaining uncertainty
`check_eligibility(profile)`	Returns eligible / near-miss / ineligible with rule trace	Pure function of YAML rules
`rank_schemes(results)`	Orders by value and effort; returns top 1 plus queue	Implements anti-overload
`build_checklist(scheme, profile)`	Documents needed, gaps, where to get each	Includes issuing office
`preflight_check(scheme, records)`	Name/DOB mismatch and dependency-chain checks	Returns fixes in order
`decode_status(scheme, status_text)`	Ranked likely causes and actions	Table-driven, sourced

Tool	Purpose	Notes
`prefill_form(scheme, profile)`	Generates PDF/JSON packet	Aadhaar masked or omitted
`schedule_followup(case, when, text)`	Creates a reminder	Respects send-rate limits
`request_confirmation(case)`	Builds the summary and payload hash	Sends the confirmation prompt
`submit_application(case, confirmation_id)`	Finalises the packet	**Only the backend can call this**, after a valid matching confirmation

8.3 Rules engine

Each scheme is a YAML file with a source and a verification date:

```
id: pm_kisan
name: PM-KISAN Samman Nidhi
source_url: https://pmkisan.gov.in
last_verified: 2026-10-19   # update only after checking official guidelines
benefit_summary: "Income support, paid in instalments"
eligibility:
  all:
    - field: owns_cultivable_land
      op: eq
      value: true
    - field: category_exclusions   # e.g., institutional landholder, income
tax payer
      op: not_in
      value: [income_tax_payer, institutional_holder]
dependencies:
  - ekyc_done
  - land_record_seeded
  - bank_aadhaar_npci_seeded
documents:
  - {name: land record, issued_by: talathi / revenue office}
  - {name: bank passbook, issued_by: bank}
status_map:
  "eKYC pending": [complete_ekyc_at_csc, check_name_match]
  "Payment Failed": [check_npci_seeding_at_bank, check_name_match]
```

Do not trust any threshold from this document or from memory. Verify every rule against the official guideline and state circulars, and record the date.

8.4 Pre-flight name matching

- Normalise: lowercase, strip honorifics and punctuation, unify common Devanagari/Latin variants (for example, vowel length, ph/f, v/w), and handle initials and surname-first order.
- Compare with token-sort and partial ratios; flag below a tuned threshold.
- Output a simple verdict (match / probable mismatch / mismatch) and the specific differing tokens.
- Evaluate on a synthetic set of 100+ name pairs with hand labels; report precision and recall.

8.5 Speech

- WAHA delivers voice notes; the backend downloads the audio and converts to 16 kHz mono WAV with ffmpeg.
- STT runs as a **separate container** with a `/transcribe` endpoint. Primary: IndicConformer (Hugging Face `ai4bharat/indic-conformer-600m-multilingual`). Fallback: Sarvam STT API.
- Pin the model revision (the model loads custom code) and check its license.
- Always read back extracted numbers and names before using them.
- Benchmark both providers on 20 real, noisy Hindi/Marathi clips before choosing the default.

8.6 Follow-ups

- Table `followups(case_id, due_at, message, status)`; a worker polls and sends via WAHA.
- Types: deadline reminder, document pickup, status re-check.
- Cap proactive messages per user and per day.

---

9. Data model (Postgres)

Table	Key fields
`users`	id, phone_hash, language, consent_at, deleted_at
`members`	id, user_id, relation, profile JSONB
`cases`	id, member_id, scheme_id, state, created_at
`checklist_items`	case_id, document, status, where_to_get
`record_checks`	case_id, record_a, record_b, verdict, differing_tokens
`confirmations`	id, case_id, payload_hash, confirmed_at, channel
`followups`	case_id, due_at, message, status
`audit_log`	ts, user_id, tool, args_redacted, result_summary
`messages`	user_id, direction, text_redacted, ts, wa_message_id

Case states: `draft → preflight → ready → awaiting_confirmation → packet_issued → tracking → resolved`.

---

10. Deployment stack

10.1 Infrastructure

- **GCP Compute Engine VM** running Ubuntu with Docker and **EasyPanel** (self-hosted PaaS with domains and automatic SSL).
- Reserve a **static external IP**; point DNS records at it.
- Firewall: allow 80/443 (and 22 from your IP only). Do **not** open database, Redis or WAHA ports publicly.
- Sizing guidance: the STT model needs real memory. Use a separate, larger VM for the STT service if the main VM is small, or switch the default STT to Sarvam. Measure CPU latency first.
- Daily automated disk snapshot or `pg_dump` to a GCS bucket.

10.2 EasyPanel services

Service	Image / source	Public?	Notes
`waha`	`devlikeapro/waha`	No (private network; protected dashboard)	Volume for session data; `WHATSAPP_DEFAULT_ENGINE=NOWEB`
`api`	Your repo (Dockerfile)	Yes, `api.yourdomain`	Webhook endpoint, website API
`worker`	Same image as `api`, different command	No	Agent jobs and follow-ups
`stt`	Your STT container	No	Needs memory; optional GPU
`postgres`	EasyPanel Postgres	No	Persistent volume, backups
`redis`	EasyPanel Redis	No	Password set

Set WAHA's webhook URL to the `api` service's internal address, and keep a shared secret header checked by the backend.

10.3 Environment variables (names are examples)

```
DATABASE_URL, REDIS_URL
ANTHROPIC_API_KEY (or chosen LLM key)
WAHA_URL, WAHA_API_KEY, WAHA_WEBHOOK_SECRET
STT_URL, SARVAM_API_KEY (fallback)
PII_HASH_SALT, APP_ENV, ALLOWED_ORIGINS
OPERATOR_AUTH_SECRET
```

Store secrets in EasyPanel's environment settings, never in the repo.

10.4 Vercel website

- Next.js app deployed from Git; environment variable `NEXT_PUBLIC_API_URL` points to the API domain.
- **Pages:** (1) landing page with a `wa.me` click-to-chat link and QR code; (2) operator dashboard (cases, stuck statuses, follow-up queue, one-click summaries) behind login; (3) transparency page listing each scheme's rules, source link and last-verified date,

plus the evaluation report.
- The site never holds secrets; it calls the backend API, which enforces authentication and CORS.

10.5 CI/CD

- GitHub repo with a Dockerfile per service; EasyPanel deploys on push to `main`.
- GitHub Actions runs pytest (rules, matcher, confirmation gate) on every push; deploy only if tests pass.

---

11. Security and privacy

- **Consent first:** the first message explains what is collected and why; store consent timestamp.
- **No Aadhaar numbers:** never request or store them. Matching needs only names and dates of birth.
- **Phone numbers hashed** in analytics tables; redact PII in logs.
- **Delete on request:** a "delete my data" command removes the user's records and generated files.
- **Retention:** auto-delete generated PDFs after a short period.
- **Access control:** operator dashboard login; API validates the WAHA webhook secret.
- **Prompt-injection safety:** the LLM can only call whitelisted tools with validated arguments; tools never accept free-form commands.
- Check India's current data-protection requirements before making any compliance claim.

---

12. Evaluation plan

Create **30 synthetic personas** with hand-labelled correct outcomes and run them automatically.

Metric	Target
Eligibility accuracy (vs. hand labels)	Report exact number
Questions asked per persona (mean)	5 to 7

Metric	Target
Name-mismatch detection precision/recall	Report on 100+ pairs
Status decoder top-3 cause hit rate	Report on a labelled set
STT word error rate on names/numbers (Hindi, Marathi)	Report per provider
Confirmation gate bypass attempts	0 successful

Publish the results on the transparency page.

---

13. Scope and build plan

Schemes (8 to 10): PM-KISAN as the flagship; plus an old-age/widow pension, Ayushman Bharat, a scholarship, a state women's scheme, and one or two Maharashtra schemes. Verify each against official sources.

Languages: Hindi and Marathi, with text fallback.

Build order

1. Rules engine + tests + eval personas
2. WAHA up in EasyPanel; webhook; echo bot
3. Agent loop with tools (text only)
4. Pre-flight checker and status decoder
5. Checklist + PDF packet + confirmation gate
6. Voice: download, convert, STT, read-back
7. Follow-up scheduler
8. Vercel site and operator dashboard
9. Eval report and demo polish

Cut order if time runs short: TTS replies, then the operator dashboard polish, then extra schemes. Never cut voice input, the pre-flight checker or the confirmation gate.

---

14. Demo script (about 3 minutes)

1. **Hook (20 s):** show the 48 lakh PM-KISAN figure and the "scheme overload" finding.

2. **Live voice note (60 s):** a Marathi voice note about stopped payments; Saathi asks for the portal status.
3. **Pre-flight catch (40 s):** "Sunil/Suneel" mismatch flagged with the fix and where to go.
4. **Confirmation gate (30 s):** the summary appears; show that editing the payload invalidates the confirmation.
5. **Follow-up and operator view (20 s):** reminder scheduled; dashboard shows the case.
6. **Numbers (10 s):** eval accuracy and questions-per-persona.

Prepare a pre-recorded fallback video in case the network or WhatsApp session fails.

---

15. Risks and mitigations

Risk	Mitigation
WhatsApp number banned (unofficial gateway)	Dedicated SIM, low volume, reply-only, spare number ready, gateway abstraction, pre-recorded demo
WAHA session drops mid-demo	Persistent session volume, health check, reconnect runbook
Marathi STT errors on names/numbers	Read-back confirmation, dual-provider STT, text fallback
Wrong rule or outdated threshold	Source + last-verified date on every rule; manual verification before demo
Single VM failure	Backups, redeploy runbook, demo video
Webhook duplicates/races	Redis dedupe and per-user locks
LLM drifts into deciding eligibility	Rules in code; tests that fail if tool output is ignored
Over-claiming	Present causes as "likely"; link official grievance channels

---

16. Items to verify before building

- WAHA free-image behaviour: voice-note media delivery, supported message types, env var names for API key and dashboard auth.
- IndicConformer license, memory use and CPU latency for Marathi.
- Every scheme rule and status meaning against official portals.
- Current Sarvam rate limits and credits.
- Data-protection obligations for storing phone numbers and household details.

---

17. Sources consulted

- The Wire: eKYC failures leave millions of farmers cut off from PM-KISAN payments (thewire.in)
- LibTech: PM-KISAN exclusion analysis, Alluri Sitaramaraju district (libtech.in)
- CEGA: Government Scheme Overload (cega.berkeley.edu)
- Ideas for India: personalised information and uptake of emergency benefits (ideasforindia.in)
- Indus Action: Karnataka guarantee schemes impact evaluation (indusaction.org)
- WAHA deployment notes (railway.com/deploy/waha-api); WAHA project on Docker Hub (`devlikeapro/waha`)
- Sarvam AI documentation (docs.sarvam.ai); AI4Bharat IndicConformer on Hugging Face

Some figures come from journalism and NGO analysis, not official statistics. Re-check them before putting them on a slide.